

le titre porte sur Software Integrity and Supply-Chain Security. Ton « gap between git model etc » est un peu en deca donc tu ferais mieux de valoriser securite et sureté de la supply chain (même chose pour intro), d'autant que « the distance between what Git's reference model technically guarantees and what producers and consumers of public repositories practically assume. » c'est peu compréhensible

CONCLUSION AND PERSPECTIVES

This thesis set out to measure a specific gap: the distance between what Git's reference model technically guarantees and what producers and consumers of public repositories practically assume. Three empirical studies, all grounded in the Software Heritage archive as the shared observational substrate, probed that gap at progressively finer resolution—from the named release labels that package managers and continuous integration pipelines treat as stable anchors, through the commit histories those labels ultimately point into, and finally into the files that contained credentials and that developers occasionally attempt to erase from those histories. This concluding chapter draws together what the three studies establish, synthesises their cross-cutting implications, and traces the work that remains.

6.1 Summary of the Thesis

une phrase introductive

6.1.1 Mutable Tags and Release Integrity

Chapter 3 reports the first ecosystem-scale study of Git tag alterations, covering 328.4 M repositories over a nine-year observation window [52]. Tag alteration affects 0.41% of all tagged repositories examined; among moves, 68.15% change the source tree behind the tag name without changing the version string visible to consumers. The pattern is not confined to obscure projects—the probability that a given repository has been altered rises with visibility—and its stakes are concrete: the Nixpkgs cross-analysis documents hash-mismatch reproducibility failures, and the tj-actions/changed-files attack is independently confirmed in the dataset as 346 simultaneous tag moves on 14 March 2025 [21].

6.1.2 Altered Histories and Repository Trust

Chapter 4 detects 12.5 B history alteration events across 1.10% of examined origins, including rewrites on the default branch (11.39 % of root-cause commits) [50]. A taxonomy of directory-content changes (DIR), metadata-only changes (META), and branch relocations separates alterations that change source trees from those that corrupt audit trails invisible to file-hash checks. Among DIR alterations, alterations of files whose names suggested credentials and retroactive license changes appear at scales large enough to study systematically. The chapter also introduces *GitHistorian*, which operationalises alteration detection on the Software Heritage graph for maintainer and auditor use [51].

6.1.3 Secret Removal and the Limits of Repository-Level Remediation

Chapter 5 asks whether files whose names suggested secrets and that were removed through history rewriting still contain recoverable, potentially usable credentials. The implemented pipeline recovers archived blobs from the second-study corpus, scans them with two independent detectors, and links findings to repository provenance; 9.18 % of 281 728 downloaded blobs yield at least one finding, and 86.4 % of private-key blobs flagged by gitleaks pass Tier A offline syntactic validation. Whether those credentials remain active is not yet established: Tier B provider-assisted validation and Tier C network checks await provider authorisation. The completed contribution is the detection infrastructure, the Tier A denominators, and the responsible disclosure framework that defines how the active-credential campaign could proceed.

6.2 What the Three Studies Establish About Repository Integrity

6.2.1 Mutability Is the Norm, Not the Exception

The picture that emerges from the two completed empirical studies is not one of rare, pathological alterations confined to a handful of poorly maintained repositories. Neither tag alteration nor history alteration is concentrated in the long tail of obscure or abandoned projects; if anything, the data point in the opposite direction: more visible and more actively maintained repositories are more likely to have experienced at least one alteration in both studies.

This finding challenges the intuition that history rewriting is primarily a practice of projects where development discipline is low. The most plausible explanation is that active development demands active maintenance, and maintenance includes both legitimate history cleanup and the automated contributions of dependency-update bots—both of which generate measurable alteration events in the archive. The normalisation of rewriting on pull-request branches may also create cultural tolerance that extends to default branches, which is the mechanism behind the non-negligible share of history alteration events on main.

tu donnes l'impression de dire de nouvelles choses alors que la conclusion devrait clore ...
d'autant que tu répètes des propos déjà tenus, pourquoi ~~88~~ sont-ils pas en 6.1.1-6.1.3 !

The practical consequence is direct: any downstream system that treats the current state of a public repository as the definitive and permanent record of what was committed is **operating on an assumption that the empirical data do not support**. This does not mean that open-source repositories are unreliable as software inputs. It means that the specific stability property that downstream consumers often implicitly require—that a reference resolved today will resolve to the same content as the same reference resolved yesterday—is not enforced by the version control system and is only intermittently satisfied in practice. *au final, on a l'impression que ce que tu as fait ne sert à rien*

6.2.2 Independent Archival as a Prerequisite for Integrity Measurement

Both empirical studies depend structurally on a property that the forge itself does not supply: the preservation of historical repository states after the repository owner has overwritten them [3]. Without an independent archive, the comparison that identifies an alteration would be impossible—the forge presents only the current state—so the snapshot-pair methodology is not merely a technical choice; it is the only way to observe these phenomena at a large scale. The implication extends beyond the specific cases reported here: integrity evidence about public repositories lives in independent archives, not in the forge, and the visibility of integrity failures is therefore contingent on archival coverage, archival frequency, and the research infrastructure available to query archived data. Software Heritage currently provides the only infrastructure capable of supplying that audit trail at the scale at which modern software supply chains operate, and its preservation function should be understood as an active component of software infrastructure rather than as a passive repository of historical curiosities. *que retenir de ta thèse ?*

6.2.3 The Remediation Gap: Repository Action vs. Credential Action

The secret-suppression case study in Chapter 4 and the detection pipeline in Chapter 5 together articulate a specific instance of a broader pattern: repository-level interventions do not substitute for authority-level interventions when the consequence of an exposure lies outside the repository. A force-pushed rewrite succeeds on the forge—the credential is no longer visible in the current branch state—but it does not close the credential’s validity at the provider, it does not retract the original content from the archive, forks, or mirrors, and the exposure window began at the original push, not at discovery. Chapter 5 establishes the detection end of this gap empirically; the severity end—whether the underlying credentials remain active on any network—**remains open, pending the Tier B and Tier C validation campaign**. *que retenir de ta thèse ?*

The same structural gap appears in milder forms across the other studies: tag alterations that affect downstream package builds, and license alterations that retroactively change the terms under which older versions were released, are both invisible to tools that observe only the current repository state [58]. The common thread is that forge-visible evidence is insufficient for integrity arguments that concern the history of what was made public and when. Closing that gap requires either enforcing stability at the forge level—through immutable releases, signed

refs, immutable audit logs, or equivalent mechanisms—or maintaining independent archives that can supply the comparative view the forge no longer holds.

6.2.4 Contributions to the Field

Taken together, the three studies deliver what Section 1.5 promised prospectively: the first ecosystem-scale measurements of tag alteration, commit-graph history alteration, and archived credential detection; the snapshot-pair framework as a general instrument for repository mutability research; *GitHistorian* as a practitioner-facing query tool; and the staged credential validation protocol as a reusable template for responsible research on sensitive archival content. The open empirical gap is narrow but significant: Study 3 establishes that removed credentials are detectable and syntactically valid in archived content, but not whether they remain active—the question the Tier B and Tier C campaign is designed to answer. Section 6.3 develops the resulting diagnosis for platforms, maintainers, and downstream consumers.

6.3 Implications for Software Supply-Chain Security

6.3.1 Implications for Repository Hosting Platforms

Hosting platforms occupy a structurally important position in the integrity picture: they are the interface through which alterations occur, and they currently represent the last point at which evidence of an alteration can be captured before it is overwritten by the next push.

For platforms that serve as dependency sources for package managers and CI infrastructure, the tag-alteration findings argue for notifications when a reference changes its target after initial publication. The scope of the problem extends to lightweight tags and annotated tags outside the formal releases interface; extending immutability guarantees, or at minimum change-detection alerts, to tags that are explicitly marked as release-stable would narrow the window for both accidental and malicious retargeting. GitHub’s opt-in immutable releases feature [24] is the most direct existing response, but it requires explicit opt-in, it applies only to one platform’s release abstraction, and it is not truly immutable.

For platforms that host repositories subject to audit requirements, the history-alteration findings motivate retention of superseded branch states when force-pushes overwrite them. Some platforms implement branch protection rules that prevent force-pushing entirely; where such rules are unavailable or not applied, independent archival remains the only mechanism that can supply a before-and-after comparison. Platforms could reduce reliance on external archival by maintaining append-only logs of ref-update events—tag creation, retargeting, deletion, and force-pushes—each entry recording reference names and associated commit hashes, including the prior target when a reference moves, optionally anchored with cryptographic timestamps or Merkle proofs to avoid tampering evidence. Logging hashes rather than tree contents would

surface alterations even when maintainers rewrite history to remove sensitive files, without re-exposing removed material through the audit mechanism itself.

For platforms that receive pushes containing secrets, the findings from Chapter 5 are not enough to argue whether platforms should strengthen their protocol to invalidate secrets or to cancel the push since we cannot yet tell what proportion of secrets are still valid.

6.3.2 Implications for Developers and Maintainers

The central practical message for developers and maintainers is that credential exposure has one security requirement and one optional hygiene step.

The required action is to rotate the credential: revoke the token, replace the key, reset the password, or otherwise render the exposed value unauthorised. This closes the exposure, because the exposure window began at the original push and the archive—or any clone, fork, or mirror—may already hold the leaked commit. Repository cleanup is optional: rewriting history, filtering past commits, or simply pushing a new commit that removes the value from the current tree all serve the same purpose once the credential is invalid—they keep the advertised repository state tidy and may satisfy organisational policy, but they do not invalidate the leak and they do not retract content from existing copies. After rotation, a stale credential that remains in commit history is not, by itself, an ongoing security problem; the failure mode this thesis documents is performing repository cleanup without rotation, which leaves the credential active.

For maintainers whose projects are dependency sources, the tag-alteration findings argue for discipline around tag creation. Tags should be published only when the release they designate is final; force-updating a published tag should be treated as an exceptional event requiring explicit communication, not as a routine correction mechanism. Signed tags or platform-level immutable release objects provide a stronger stability guarantee for consumers who require it. For maintainers of popular repositories in particular, the data show that visibility is not a protection against alteration: the probability of at least one alteration event rises, not falls, with star count.

6.3.3 Implications for Downstream Consumers and Auditors

Downstream consumers—developers who depend on third-party packages, build systems that resolve version specifications, and auditors who verify supply-chain integrity—face a structural blind spot: they typically observe repositories only through the current forge state, and the current state contains no evidence of prior alterations that have since been overwritten.

The practical implication for build-critical dependency resolution is that version label pinning without content-hash verification provides a weaker integrity guarantee than what the conventions of most dependency management tools suggest. A lockfile that records a tag name or a symbolic version string is guaranteed only by the social convention of the repository maintainer; a lockfile that records a content-addressed object identifier is guaranteed by the cryptographic

structure of Git itself. For any domain where bit-for-bit reproducibility is a requirement rather than a convenience—scientific software, regulatory-compliant systems, infrastructure tooling—content-addressed pinning is the appropriate default.

For auditors who need to verify what code was present in a repository at a specific point in the past, the findings argue for consulting independent archives rather than relying on the current forge state. Software Heritage provides the archival substrate for exactly this query, and GitHistorian provides a practical interface for it. If an organisation’s audit policy treats repositories as the authoritative record of what was built when, the measurements in this thesis show that such a policy has systematic blind spots whenever history alterations have occurred between the build time and the audit time.

6.4 Limitations of the Thesis

The three studies share a set of cross-cutting limitations that arise from the observational methodology.

Lower bounds throughout. All prevalence figures reported in this thesis are conservative lower bounds on the true scale of the phenomena they measure. An alteration that occurs and is reversed between two consecutive archival visits leaves no trace in the snapshot-pair comparison. Multiple alterations between two visits are merged into one observable event. Short-lived alterations, rapidly corrected mistakes, and alterations to repositories archived only once or never archived are entirely invisible to both empirical studies. The direction of the bias is unambiguous: undercounting, not inflation.

Observational, not causal. All three studies operate on archival traces: the observable effects of alterations that have already occurred, recorded by a third party without access to the repository owners’ intentions or workflows. Intent cannot be inferred from alteration type alone. A force-pushed main branch may reflect a merge, a routine cleanup, a targeted supply-chain attack, or the automated behaviour of a dependency-update bot. A removed credential file may reflect careful rotation, confused remediation, or an attempt to conceal evidence of a compromise account. The studies characterise what happened and at what scale; they do not establish why.

Single version control system. All analyses are restricted to Git repositories. No claim is made about other distributed version control systems, and the specific properties of Git’s reference model—the distinction between lightweight and annotated tags, the semantics of force-push, the absence of append-only guarantees on branches at the protocol level—are central to the phenomena under study. Some other version control systems may support non-destructive history updates that retain previous states by design, which would make the analogous measurement structurally different.

Archival coverage and frequency. The measurements depend on the archival frequency and coverage of Software Heritage. Repositories that were not archived, that were archived

only once, or whose alteration occurred in a long interval between visits are absent from the results. The studies cannot correct for these gaps; they can only state them, and their prevalence estimates should be read accordingly.

Study 3 is incomplete. The secret removal study reported in Chapter 5 presents an unfinished measurement. The detection pipeline, Tier A offline validation for private keys, and the responsible disclosure framework constitute the completed contributions; the active-credential campaign (Tier B provider-assisted validation and Tier C network checks conducted by the researcher) has not been executed. No active-rate result is available, and no claim is made about whether the credentials detected in archived content remain usable on any network. Until the validation campaign is completed, the central question that motivates Chapter 5—whether developers who rewrote history to remove a credential also rotated that credential—remains empirically unanswered. **This is the most significant open limitation of the thesis as a whole.**

limitation no. need to be finalized

6.5 Perspectives

cela reste très proche de release tag, commits and secrets

6.5.1 Improved Observability of Repository Mutability

The measurement programme reported in this thesis is retrospective: it queries a historical archive to identify alterations that occurred in the past and does not provide real-time awareness to the actors who could act on that information. Moving toward continuous observability would substantially increase the practical value of the findings.

The most direct direction is an online monitoring service that applies the snapshot-pair comparison framework as new Software Heritage visits are completed, rather than as a single batch analysis over a frozen dataset. Such a service could notify package registry maintainers, dependency management tools, or individual developers when a repository they depend on has undergone a tag alteration or history rewrite, enabling timely remediation rather than retrospective discovery.

A complementary direction is to consume forge-native event streams directly. Where platforms expose real-time push, force-push, and ref-update events through APIs or webhooks, subscribing to those streams would reduce the latency between an alteration and its detection below the archival visit interval, which can span days or weeks for less frequently crawled origins, and could feed the append-only ref logs recommended in Section 6.3.1.

A third direction concerns breadth. Both completed studies target Git repositories. Software Heritage archives Mercurial, Subversion, and other version control systems as well, and the snapshot-pair idea is transferable in principle, though each system’s mutability model differs and would require an adapted detection framework. Whether the alteration patterns observed in Git are specific to its reference semantics or reflect broader development norms is an open empirical question.

6.5.2 Automated Detection and Alerting

GitHistorian represents an initial step toward practical alteration detection, but it requires practitioners to supply the repository URL, to manage the local database and to download the complete database of Chapter 4’s results. A natural next step is a hosted service—integrated into CI/CD platforms, package registries, or software bill of materials tooling—that tracks the repositories a project depends on and reports alteration events without requiring per-user setup.

For the tag-alteration dimension, this could take the form of a check that runs as part of dependency update workflows: before committing a lockfile update, the tool verifies that no tag in the dependency tree has been retargeted since the last resolution and surfaces the information if one has. For use cases focused on security, a CI gate that queries the alteration record on each dependency incorporation would flag unexpected rewrites before they reach the consuming project. The practical challenge is scale: maintaining a queryable alteration database for all packages in a major registry requires infrastructure investment that individual practitioners cannot bear alone. Partnerships between tool builders and the Software Heritage infrastructure, or the publication of precomputed alteration indices alongside each Software Heritage graph release, could be a path forward.

For the history-alteration dimension, a lightweight integration that checks each newly added dependency against GitHistorian before merge would provide an early warning for organisations whose threat model includes supply-chain tampering through commit-graph rewriting. Automating this check in CI would bring detection latency closer to zero without requiring manual attention from developers or auditors.

6.5.3 Developer Behavior and Remediation Practices

The measurements in this thesis establish that repository alterations happen at scale and that some of them have security-relevant file-level footprints. They do not establish what developers do before and after those alterations, and in particular whether the developers who rewrote history to remove a credential also rotated that credential.

The most direct open task is completing the active-credential campaign for Chapter 5. The Tier A offline denominators for private keys are now available; extending the same structural filtering to the remaining credential families and initiating provider contact for Tier B validation are the next required steps. The outcome of that campaign—whether a detectable fraction of removed credentials remains active, and for which credential families—would provide the most direct empirical answer to the question of whether history rewriting is used as a substitute for rotation.

Beyond the immediate completion of Study 3, a qualitative research programme could investigate developer understanding of the distinction between repository-level and credential-level remediation. Prior work on secret-management practices [6] establishes the recommended distinction between rotation and repository cleanup and notes divergences from it in observed developer

behaviour. A natural extension is to study developers who have experienced a credential-leakage incident directly, asking whether the rotation/cleanup distinction was understood, what guidance influenced their response, and whether platform-provided scanning and revocation workflows led them to rotate rather than rely on cleanup alone.

A temporal analysis of detected secrets would complement this behavioural work without requiring the Tier B or Tier C campaign to proceed. Pairing the archival timestamp of the first observation of a credential-bearing commit with the timestamp of the history alteration that removed it provides an exposure window for each detected secret. Short windows might suggest that platform-side scanning triggered a rapid response; long windows might indicate that the exposure was discovered through an incident rather than a proactive check. Whether quick remediation correlates with no rotation—the scenario in which a developer notices the problem late, rewrites immediately, but omits the provider step because the credential appears not to have been exploited—is precisely the question that the combination of temporal analysis and active-rate measurement would answer.

6.6 Final Remarks

que dis tu de nouveau ? repetition?

This thesis began with the `tj-actions/changed-files` incident—hundreds of version tags retargeted overnight, thousands of pipelines executing attacker code while believing they had pinned trusted versions—striking not because it was novel, but because it was visible and well-documented. The three studies reported in this thesis ask how far the underlying vulnerability extends in the ecosystem when no adversary is present—when maintainers move tags to correct mistakes, rewrite histories to apply patches, and remove committed credentials as part of ordinary incident response.

The answer, measured across tens of millions of repositories over a decade of archival observation, is that the gap between Git’s object-level integrity guarantees and the stability assumptions of the systems that consume public repositories is endemic rather than exceptional. Tag names move regularly. Commit histories are routinely rewritten, including on default branches of popular projects. Files whose names suggested credentials are altered from those histories at scale, and the removed content remains recoverable from the archive regardless of what the repository record shows. None of this requires an adversary. It is the ordinary behaviour of ordinary maintenance, aggregated to the scale at which modern software supply chains operate.

The diagnosis the three studies collectively support is precise: the reference layer of version control does not enforce the stability contract that the consumption layer depends on, and the evidence of violations disappears from the forge the moment an overwriting push arrives. Closing that gap requires infrastructure that persists evidence independently of the repository owner’s choices—content-addressed pinning at the consumer end, and independent archival at the observation end. Software Heritage currently provides the only infrastructure that makes

this observability possible at ecosystem scale, and the studies reported in this thesis demonstrate that its function as an integrity witness is as important as its function as a preservation service.

In a software supply chain built on mutable references, integrity is not a property of the repository as the forge presents it, it is a property of the record that outlasts the last push.